Lab 16 — Web Application Firewall (WAF) with ModSecurity

You will deploy Apache + ModSecurity + the OWASP Core Rule Set, then watch it block real attack payloads.

Run on https://killercoda.com/playgrounds/scenario/ubuntu

The nginx ModSecurity connector is not packaged in the Ubuntu 22.04 archive, so this lab uses the Apache module `libapache2-mod-security2`, which is available on both Ubuntu 22.04 and 24.04.

> **Required software (free, apt):** - `apache2` - `libapache2-mod-security2` - `modsecurity-crs`

Step 1 — Install

```
apt update && apt install -y apache2 libapache2-mod-security2 modsecurity-crs
```

Installing `libapache2-mod-security2` enables the Apache `security2` module automatically.

Step 2 — Enable the rule engine and load the CRS

```
cp /etc/modsecurity/modsecurity.conf-recommended /etc/modsecurity/modsecurity.conf
```

```
sed -i 's/^SecRuleEngine DetectionOnly/SecRuleEngine On/' /etc/modsecurity/modsecurity.conf
```

```
# Apache's security2 config already does: IncludeOptional /etc/modsecurity/*.conf
```

```
# Add a file (loaded after modsecurity.conf) that pulls in the OWASP CRS.
echo 'Include /usr/share/modsecurity-crs/owasp-crs.load' > /etc/modsecurity/owasp-crs.conf
```

```
apachectl configtest && systemctl restart apache2
```

Step 3 — Confirm CRS is loaded

```
apachectl -M 2>/dev/null | grep security2
```

```
grep -rh "^SecRule" /usr/share/modsecurity-crs/rules/*.conf | wc -l
```

You should see the `security2_module` loaded and hundreds of rules on disk.

Step 4 — Fire common attack payloads

```
curl -s -o /dev/null -w "%{http_code}\n" "http://127.0.0.1/?id=1' OR '1'='1"

curl -s -o /dev/null -w "%{http_code}\n" "http://127.0.0.1/?q=<script>alert(1)</script>"

curl -s -o /dev/null -w "%{http_code}\n" "http://127.0.0.1/?file=../../etc/passwd"

curl -s -o /dev/null -w "%{http_code}\n" "http://127.0.0.1/?cmd=;id"
```

Expect `403` for each — CRS recognises SQLi, XSS, path traversal, and command injection.

Step 5 — Read the audit log

```
tail -50 /var/log/modsec_audit.log

grep -E "Matched|attack" /var/log/modsec_audit.log | tail
```

Each event records the rule ID, severity, and the offending payload.

Step 6 — Tune a false positive

If a legitimate request is blocked, exempt the specific rule for that path:

```
echo 'SecRule REQUEST_URI "@beginsWith /admin/edit"
"id:1000,phase:1,pass,nolog,ctl:ruleRemoveById=941100"' \
  >> /etc/modsecurity/modsecurity.conf

systemctl reload apache2
```

Free online tools

- **OWASP CRS docs** — https://coreruleset.org
- **ModSecurity reference** — https://github.com/SpiderLabs/ModSecurity/wiki
- **Cloudflare WAF (free tier)** — https://www.cloudflare.com
- **AWS WAF managed rules (pay-as-you-go)** — https://aws.amazon.com/waf/
- **Wafw00f** — WAF detection: https://github.com/EnableSecurity/wafw00f

What you learned

- A WAF is layer-7 inspection on top of an HTTP server.
- The OWASP CRS gives instant coverage of OWASP Top 10 with no custom rules.
- How to triage rule blocks and selectively turn off rules per path.